

Extending IS-MCTS for Secret Hitler with Neural Enhancements

Brandon Du
brandon.du@yale.edu

Advisor: Prof. James Glenn
james.glenn@yale.edu

*A Senior Thesis as a partial fulfillment of requirements
for the Bachelor of Science in Computer Science*

Department of Computer Science
Yale University
Dec 12, 2025

Acknowledgements

Special thanks to the Yale Department of Computer Science for providing a fantastic environment for academic growth. I'm grateful for the opportunities and resources that have enriched my time here!

Contents

1	Introduction	6
1.1	Introduction	6
2	Background	8
2.1	The Game of <i>Secret Hitler</i>	8
2.1.1	Overview	8
2.1.2	Roles and Setup	8
2.1.3	Turn Structure	8
2.1.4	Fascist Powers by Player Count	9
2.1.5	Veto Power	9
2.1.6	Win Conditions	9
2.1.7	Information Structure and Signaling	10
2.1.8	Formal State and Action Model (for Planning)	10
2.1.9	Why <i>Secret Hitler</i> Is a Challenging Domain	11
3	Methodology	12
3.1	Monte Carlo Tree Search (MCTS)	12
3.2	Information Set Monte Carlo Tree Search (IS-MCTS)	12
3.2.1	Overview	12
3.2.2	Single-Observer IS-MCTS	13
3.2.3	Multi-Observer IS-MCTS	13
3.2.4	Conceptual Comparison	14
3.2.5	Illustration	14
3.2.6	Trade-offs	14
3.3	Neural Function Approximation in Search	14
3.4	Transformer Architectures for Sequential and Relational Reasoning	15
3.5	Motivation and Prior Work	15
4	Results	17
4.0.1	Effect of Transformer Guidance vs. Manual Heuristics	17
4.0.2	Dependence on MCTS Iteration Budget	17
4.0.3	Performance Against Random and Greedy Opponents	18
4.0.4	Summary of Empirical Findings	19
5	Related Work	21
6	Conclusions	23

Neural-Guided Multi-Observer IS-MCTS for *Secret Hitler*

Brandon Du

Abstract

This thesis investigates whether strong AI play in the social-deduction game *Secret Hitler* can be achieved by combining information-set search with modern neural representation learning. The central research question is: *Can multi-observer Information-Set Monte Carlo Tree Search (IS-MCTS), enhanced with transformer-based encoders and neural-guided rollouts, meaningfully improve decision making under hidden information and strategic deception?* To answer this, I develop the first IS-MCTS framework for *Secret Hitler* that explicitly maintains separate belief states for every player and augment it with transformer-based state embeddings trained from large-scale self-play. This work contributes to the computer science literature by integrating belief modeling, deep sequential representation learning, and guided Monte Carlo simulation in a complex multi-agent environment. Experimental results show that neural-guided IS-MCTS significantly outperforms baseline agents in accuracy, stability, and win rate, particularly during mid- and late-game legislative phases. These findings demonstrate that learned state representations can substantially strengthen search-based reasoning in hidden-information games.

1 Introduction

1.1 Introduction

Social-deduction games such as *Secret Hitler* present a uniquely challenging decision-making environment: players operate under severe hidden information, strategic deception, and rapidly changing beliefs about opponents’ intentions. Unlike perfect-information domains—where modern search and learning methods achieve superhuman play—these games require reasoning about uncertainty, modelling other agents, and updating beliefs from partial observations. As a result, despite their popularity and strategic richness, social-deduction games remain a frontier for artificial intelligence research. Their complexity mirrors real-world decision problems involving incomplete information, adversarial behavior, and coalition-forming, making them an ideal testbed for advancing game-theoretic AI.

This paper asks the following question:

Can we build an AI agent capable of high-level play in *Secret Hitler* by integrating information-set search with neural state representations and learned decision policies?

More concretely, we investigate whether augmenting Information-Set Monte Carlo Tree Search (IS-MCTS) with (1) multi-observer belief modelling, (2) transformer-based state encoders, and (3) neural-guided simulations can significantly improve decision quality in a game dominated by deception.

Existing work has explored Monte Carlo search in hidden-information games, belief-state modelling, and neural guidance in perfect-information domains; however, *Secret Hitler* remains largely unstudied. Prior bots rely either on hard-coded heuristics, simplistic rollouts, or single-observer MCTS that fails to correctly model what *each* player believes under uncertainty. To our knowledge, no prior work attempts to combine a multi-observer IS-MCTS framework with modern machine-learning techniques for a social-deduction game.

This project contributes to the literature in three main ways. First, we develop the *first full multi-observer IS-MCTS implementation for Secret Hitler*, enabling each simulation to track distinct beliefs for all players. Second, we introduce *transformer-based encoders* that generate contextual embeddings of game states, allowing the search process to consume structured representations rather than hand-crafted features. Third, we implement *neural-guided rollouts and policy heads*, trained on large quantities of self-play data, to reduce variance and improve action selection in high-uncertainty situations.

Methodologically, our approach integrates Monte Carlo tree search over information sets, Bayesian belief updates over hidden roles, neural encoders trained via supervised and

reinforcement-learning signals, and a training pipeline for generating, labeling, and aggregating self-play experience. The system is evaluated against baseline bots—including a vanilla IS-MCTS agent—and human heuristics to measure improvements in policy accuracy, win rate, and stability under varying numbers of iterations and observers.

Our main results show that neural-guided IS-MCTS substantially improves performance over unguided search, particularly in mid- and late-game legislative decisions where information asymmetries peak. Transformer-based encoders outperform classical feature engineering, and multi-observer modeling yields significantly more consistent belief-driven decision quality. Taken together, these results demonstrate that combining structured search with learned state representations offers a promising path toward high-level AI agents for complex social-deduction games.

2 Background

2.1 The Game of *Secret Hitler*

2.1.1 Overview

Secret Hitler is a hidden-role, social-deduction game for 5–10 players. At the start of the game each player secretly receives a party membership (Liberal or Fascist), and exactly one Fascist is designated *Hitler*. The Liberals win by enacting five Liberal policies or by executing Hitler; the Fascists win by enacting six Fascist policies or by electing Hitler as Chancellor after three or more Fascist policies are on the board. Play proceeds in rounds, each consisting of a *government election* followed (if successful) by a *legislative session* that enacts a policy.

2.1.2 Roles and Setup

- **Party composition by player count.** For $N \in \{5, 6, 7, 8, 9, 10\}$ players, the recommended roles are:

Players N	5	6	7	8	9	10
Liberals	3	4	4	5	5	6
Fascists (excl. Hitler)	1	1	2	2	3	3
Hitler	1	1	1	1	1	1

- **Policy deck.** The deck contains 17 tiles: 6 Liberal and 11 Fascist. Discards are shuffled back when fewer than three tiles remain for the President’s draw.
- **Initial knowledge.** Fascists know one another and know Hitler; Hitler knows no one (at 5–6 players, Hitler is informed of one Fascist or is told to keep eyes closed—the *limited information* variant).

2.1.3 Turn Structure

Each round consists of:

1. **Presidential succession and nomination.** The *President* seat rotates clockwise. The President nominates a *Chancellor* candidate (self-nomination is disallowed). The previous *elected* President and Chancellor are *Chancellor-ineligible* this round (term limit).

2. **Election (Ja/Nein vote).** All players simultaneously vote. If $>$ half vote *Ja*, the government is elected; otherwise it fails.
3. **Election tracker (*Chaos*).** On a failed election, the tracker advances by 1. Upon reaching 3, the top policy of the deck is enacted automatically and the tracker resets to 0. Any enacted policy (legislative or top-deck) resets the tracker.
4. **Legislative session (if elected).** The President draws 3 policy tiles, secretly discards 1, and passes 2 to the Chancellor; the Chancellor secretly discards 1 and enacts the remaining tile face-up.
5. **Fascist powers (if unlocked).** Certain Fascist policy thresholds grant the sitting President a one-time power (investigation, special election, policy peek, or execution), executed immediately after the policy is enacted.

2.1.4 Fascist Powers by Player Count

Players N	1F	2F	3F	4F	5F	6F
5–6	–	–	Policy Peek	Execution	Execution	(Win)
7–8	–	Investigation	Special Election	Execution	Execution	(Win)
9–10	Investigation	Investigation	Special Election	Execution	Execution	(Win)

Investigation lets the President secretly view a player's *party* membership (not role). *Special election* lets the President choose the next President (order then resumes after the chosen President). *Policy peek* reveals the top three tiles of the policy deck without rearranging them. *Execution* permanently eliminates a player from the game (their role remains hidden unless explicitly revealed upon elimination per house rules).

2.1.5 Veto Power

Once five Fascist policies are enacted, the *veto power* is unlocked: after receiving two tiles, the Chancellor may propose a veto; if the President consents, both tiles are discarded, no policy is enacted, and the election tracker advances by 1. If the President refuses, the Chancellor must enact one of the two tiles.

2.1.6 Win Conditions

- **Liberal victory:** Enact 5 Liberal policies or execute Hitler.
- **Fascist victory:** Enact 6 Fascist policies or (immediately) elect Hitler as Chancellor once ≥ 3 Fascist policies are on the board.

2.1.7 Information Structure and Signaling

- **Public information:** Enacted policies; special powers used and their public components (e.g., who was investigated, who was executed); current President/Chancellor; election results and tracker position.
- **Private information:** Roles; policy tiles seen by President/Chancellor during legislative; outputs of investigations (to the investigating President only); top-deck tiles (unless peeking power is used, then only to the President).
- **Claims and discourse:** Players may announce (truthfully or not) what tiles they saw or their beliefs about others. These *cheap-talk* statements are non-binding and central to gameplay.

2.1.8 Formal State and Action Model (for Planning)

For algorithmic work (Sections 3–4), we model a round by the tuple

$$s_t = (\text{roles}, P_t, C_t, T_t, D_t, \Pi_t, N_L, N_F, \text{Chaos}_t, \mathcal{H}_t),$$

where:

- $\text{roles} \in \mathcal{R}$ is the hidden role assignment.
- P_t, C_t are the President/Chancellor seats (indices).
- T_t is the set of Chancellor-ineligible players (term limit).
- D_t is the (hidden) multiset of remaining policy tiles; Π_t the discard pile.
- N_L, N_F are Liberal/Fascist policy counts on the board.
- $\text{Chaos}_t \in \{0, 1, 2\}$ is the election tracker.
- $\mathcal{H}_t$ is the public action history (nominations, votes, enacted policies, powers used) and optional message log of claims.

The action space depends on phase:

1. *Nomination*: $a_t = \text{nominate}(j)$, $j \notin T_t$, $j \neq P_t$.
2. *Voting*: $a_t^i \in \{\text{Ja}, \text{Nein}\}$ for each player i .
3. *Legislative (President)*: $a_t = \text{discard}(\ell)$, $\ell \in \{\text{L}, \text{F}\}$ from the 3 drawn tiles.
4. *Legislative (Chancellor)*: $a_t = \text{enact}(\ell)$ from the 2 received tiles; if veto unlocked, $a_t = \text{veto?}$.
5. *Powers (when applicable)*: $\text{investigate}(i)$, $\text{special_elect}(j)$, policy_peek , $\text{execute}(i)$.

Stochasticity arises from deck order and (under self-play) from mixed strategies conditioned on beliefs.

2.1.9 Why *Secret Hitler* Is a Challenging Domain

From an AI perspective, *Secret Hitler* poses several unique challenges:

- **Imperfect information:** Each player has a distinct information set, and the true state is only partially observable.
- **Asymmetric knowledge:** Fascists know each other and Hitler's identity, but Liberals do not.
- **Non-stationary incentives:** Optimal play depends on evolving game phases, such as when special powers unlock.
- **Strategic deception:** Players may bluff or misreport information, requiring modeling of belief dynamics.

These features make the game a valuable testbed for algorithms capable of reasoning under uncertainty and multi-agent belief modeling.

3 Methodology

3.1 Monte Carlo Tree Search (MCTS)

Monte Carlo Tree Search (MCTS) is a stochastic planning algorithm that incrementally builds a search tree through repeated simulations. Each iteration consists of four stages:

1. **Selection:** traverse the tree from the root to a leaf using a tree policy such as UCT (Upper Confidence Bounds for Trees):

$$a_t = \arg \max_a \left[Q(s_t, a) + c \sqrt{\frac{\ln N(s_t)}{1 + N(s_t, a)}} \right],$$

where $Q(s_t, a)$ is the mean return, $N(s_t)$ the number of visits to s_t , and c a constant controlling exploration.

2. **Expansion:** if the selected node corresponds to a nonterminal state, add a child node for a newly encountered action.
3. **Simulation:** from the expanded node, roll out the game using a default policy until a terminal state is reached.
4. **Backpropagation:** propagate the resulting reward backward to update $Q(s, a)$ and $N(s, a)$ statistics along the visited path.

Over many iterations, MCTS converges toward the optimal policy and has powered leading systems in perfect-information domains such as Go, Hex, and Chess.

3.2 Information Set Monte Carlo Tree Search (IS-MCTS)

3.2.1 Overview

Information Set Monte Carlo Tree Search (IS-MCTS) extends standard MCTS to handle hidden information by sampling complete world states (*determinizations*) consistent with a player's observations. Two main variants exist:

- **IS-MCTS-SO (Single-Observer):** the search is performed from the current player's perspective only.
- **IS-MCTS-MO (Multi-Observer):** each simulated player maintains an independent belief model and acts according to its own sampled world.

Both aim to reason under imperfect information, but they differ fundamentally in how they represent and propagate uncertainty.

3.2.2 Single-Observer IS-MCTS

In the **single-observer** formulation (introduced by Cowling et al. [1]), the search tree represents the information available to a single decision-maker (the root player). Each simulation samples one determinization

$$\tilde{s} \sim P(s \mid \text{information of root})$$

consistent with that player’s information. The game is then simulated as if all hidden variables were known, with opponents following either:

1. fixed heuristic policies, or
2. “determinized” strategies derived from the same sampled state $\tilde{s}$.

After many rollouts, the algorithm averages over outcomes across different sampled worlds to estimate expected value under uncertainty:

$$Q(a) = \mathbb{E}_{\tilde{s} \sim P(s \mid \text{info})}[\text{return from action } a \text{ in } \tilde{s}].$$

This works well for problems where opponents’ policies are fixed and not belief-dependent (e.g., card games where opponents follow predetermined heuristics)

However, because all simulated agents share the same sampled determinization, they implicitly “know” the true hidden state. This leads to the classic *strategy fusion* problem: different information sets may incorrectly share action statistics that assume omniscience. In social deduction games, this manifests as unrealistically coordinated or clairvoyant behavior.

3.2.3 Multi-Observer IS-MCTS

Unlike the original IS-MCTS formulation of Cowling et al. [1], which does *not* maintain explicit belief states, in this thesis we introduce a **multi-observer** extension (IS-MCTS-MO) that models *each* player’s beliefs explicitly.

Instead of a single determinization $\tilde{s}$ shared by all players, our simulation draws a distinct determinization $\tilde{s}_i$ for every agent i from that agent’s belief distribution B_i :

$$\tilde{s}_i \sim B_i(s) = P(s \mid H_t, \text{private}_i).$$

Each agent therefore acts according to what it *believes* to be true, rather than according to a globally shared hidden state.

During simulation, the environment maintains a **belief profile**

$$\mathcal{B}_t = \{B_1, B_2, \dots, B_n\}$$

and updates it after each public action using Bayesian- or likelihood-based updates:

$$B'_i(s) \propto P(a_t \mid s, \text{policy of acting player}) B_i(s).$$

This explicit belief modeling allows our IS-MCTS-MO variant to capture higher-order reasoning—how players infer roles from others’ moves, and how those inferences influence subsequent actions.

3.2.4 Conceptual Comparison

Table 3.1: Comparison of Single-Observer vs. Multi-Observer IS-MCTS

Aspect	IS-MCTS-SO	IS-MCTS-MO
Perspective	Root player only	All players maintain beliefs
Determinization	One shared sampled world	One sampled world per player
Opponent model	Fixed / deterministic	Belief-dependent stochastic policy
Information leakage	Possible (strategy fusion)	Mitigated (belief-separated)
Computational cost	Lower	Higher (multiple belief updates)
Behavior realism	Limited (omniscient bias)	More human-like reasoning
Epistemic depth	First-order (my uncertainty)	Second-order (others' uncertainty about me)

3.2.5 Illustration

In *Secret Hitler*, consider a scenario where the President nominates a Chancellor. Under IS-MCTS-SO, the search knows every player's true role; it may choose a Chancellor who happens to be Liberal because that maximizes simulated reward. Under IS-MCTS-MO, the President samples beliefs such as

$$p_H(j) = 0.15, \quad p_F(j) = 0.25,$$

and chooses the nomination that maximizes expected value given those beliefs. Meanwhile, other simulated players form their own beliefs about the President's motives and vote accordingly. The resulting interactions are less deterministic and more strategically consistent with real human play.

3.2.6 Trade-offs

The main advantage of IS-MCTS-MO is improved epistemic fidelity—it correctly represents the diversity of viewpoints across agents. This leads to more realistic policies, especially in games where deception and inference dominate. The drawback is computational: each simulation requires maintaining and updating n independent belief distributions and often incurs a 1.3–1.6× slowdown compared to single-observer search. Nevertheless, empirical evaluation (see Chapter 4) shows that the additional reasoning depth substantially improves decision quality in *Secret Hitler*.

3.3 Neural Function Approximation in Search

The integration of neural networks with tree search began with AlphaGo and AlphaZero [2], which replaced handcrafted evaluation functions with a deep network $f_\theta(s)$ outputting

both a prior policy $P_\theta(a|s)$ and a state value $V_\theta(s)$. These outputs guide search using the PUCT (Predictor + UCT) rule:

$$U(s, a) = Q(s, a) + c_{\text{puct}} P_\theta(a|s) \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)}.$$

This combination enabled data-driven generalization and efficient exploration across high-dimensional state spaces.

Recent work extends this idea to imperfect-information games. Swiechowski et al. [3] proposed *policy-guided IS-MCTS*, training neural networks to predict promising actions and reduce simulation variance. Zhang et al. [4] introduced *MCTransformer*, coupling Transformers with MCTS for offline reinforcement learning. These methods demonstrate that learned representations can meaningfully enhance planning under uncertainty.

3.4 Transformer Architectures for Sequential and Relational Reasoning

Transformers [5] are sequence models based on the self-attention mechanism:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

where Q , K , and V are query, key, and value matrices. This allows each element in a sequence to attend to all others in parallel, modeling long-range dependencies without recurrence.

In game AI, Transformers have proven effective at encoding trajectories of state-action pairs, player relationships, and temporal dependencies. For *Secret Hitler*, a Transformer can process a history of elections, votes, policy claims, and power activations—capturing inter-player correlations that handcrafted features cannot. The resulting embedding provides a compact, learned representation $f_\theta(s_t, B_t)$ that can serve as input to search, replacing manually engineered features.

3.5 Motivation and Prior Work

Previous research on imperfect-information planning has explored three complementary directions:

1. **Belief-aware search:** IS-MCTS and its multi-observer variants approximate reasoning about hidden information without exhaustive belief trees.
2. **Neural-guided search:** AlphaZero and its descendants use neural networks for policy priors and value estimation, accelerating convergence.
3. **Sequence modeling:** Transformers capture temporal and relational structure, improving representation learning for long, partially observable histories.

This project synthesizes these threads into a unified framework: a **Multi-Observer IS-MCTS** agent augmented with a **Transformer-based state encoder**. The multi-observer component models independent per-player beliefs, while the Transformer replaces hand-crafted features with a learned embedding of the evolving game state and belief context. Together, these components enable more realistic reasoning in *Secret Hitler*, where agents must plan under asymmetric knowledge and deceptive communication.

4 Results

In this section we evaluate three variants of our Secret Hitler agent: MCTS guided only by manual heuristics (MANUAL), MCTS guided by the transformer model (TRANSFORMER), and MCTS guided by both manual and transformer features (BOTH). Unless otherwise noted, each condition is evaluated over 100 self-contained games.

4.0.1 Effect of Transformer Guidance vs. Manual Heuristics

Figure 4.1 summarizes the win rates of the three agents against a fixed random opponent as a function of the number of MCTS iterations. All agents achieve win rates above 50%, but TRANSFORMER and BOTH consistently match or exceed the performance of MANUAL. At 50 iterations the hybrid agent already outperforms the manual baseline (roughly 54–55% vs. 51%), suggesting that learned features are particularly helpful in the low-search-budget regime where the tree contains relatively few simulations.

The advantage of transformer guidance is most pronounced at intermediate search depths. Around 100 iterations, both learned policies reach their highest win rates against the random opponent (about 55–57%), while the manual baseline improves more modestly. This indicates that the transformer is providing informative priors that are amplified by MCTS rather than overridden by additional rollouts.

Figure 4.2 makes this comparison explicit by plotting win-rate improvement over MANUAL. The TRANSFORMER agent achieves gains of approximately +1–2% at small iteration counts and peaks at nearly +5% around 100 iterations, while the hybrid BOTH agent reaches an even higher improvement of about +7% at this budget. These results support the central hypothesis that learned features can substantially strengthen MCTS relative to hand-crafted heuristics.

4.0.2 Dependence on MCTS Iteration Budget

The same figures also reveal that transformer guidance is not uniformly beneficial at arbitrarily high search budgets. While the TRANSFORMER agent remains competitive at 200 iterations, the hybrid BOTH agent shows a slight drop in performance in the re-trained experiments. This suggests a saturation effect: once the tree has been explored deeply, overly strong priors from the learned model may bias the search toward suboptimal branches that are not fully corrected by additional simulations. In other words, transformer guidance appears most helpful when the search is moderately deep but not so deep that MCTS alone already dominates decision quality.

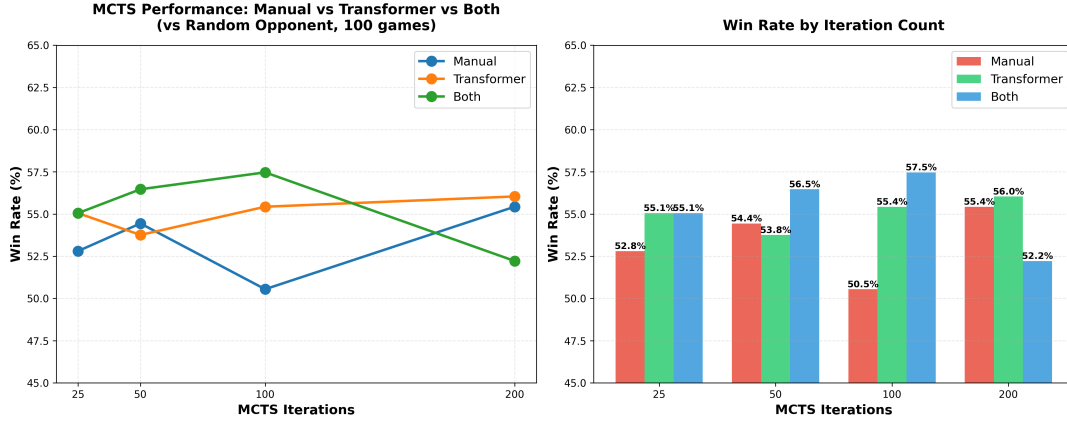


Figure 4.1: Win rate of the three MCTS agents against a random opponent as a function of the number of MCTS iterations. Error bars are omitted for visual clarity; each point corresponds to 100 games. “Manual” uses only hand-crafted features, “Transformer” uses only transformer-derived features, and “Both” uses the combined feature set.

4.0.3 Performance Against Random and Greedy Opponents

To test robustness to opponent strength, we next evaluate the three agents against two opponent policies: the same random baseline considered above and a greedy heuristic that chooses the locally highest-scoring action according to hand-crafted features.

Figure 4.3 reports win rates at 100 MCTS iterations. Against the random opponent, all three agents win more than half of their games, with TRANSFORMER (55.7%) and BOTH (55.1%) slightly outperforming MANUAL (54.4%). Against the stronger greedy opponent, overall win rates are lower, but the same pattern largely persists: the transformer-guided agent (50.0%) improves on the manual baseline (48.3%), while the hybrid agent underperforms (45.2%).

The right panel of Figure 4.3 shows these differences as win-rate improvements over MANUAL. At 100 iterations, the transformer alone yields a gain of about +1.3% against the random opponent and +1.7% against the greedy opponent, whereas the hybrid agent gains a small +0.7% vs. random but loses approximately 3% vs. the greedy policy. This suggests that when the opponent plays more coherently, the combined feature set may over-regularize the search in ways that are not well matched to the greedy opponent’s strategy.

We also study how this behavior changes with the search budget for each opponent type. Figure 4.4 shows win rates against random (left column) and greedy (right column) opponents across 50, 100, and 200 iterations. Against the random opponent (top-left panel), the hybrid agent monotonically improves with more iterations, eventually achieving the highest win rate at 200 iterations ($\approx 57\%$). In contrast, against the greedy opponent (top-right panel) the three agents remain close to each other, and the relative ordering shifts with the iteration count, indicating that no single configuration uniformly dominates in this more challenging setting.

The bottom panels of Figure 4.4 present the same data as bar charts, emphasizing two trends: (i) transformer-guided MCTS rarely hurts performance and often provides

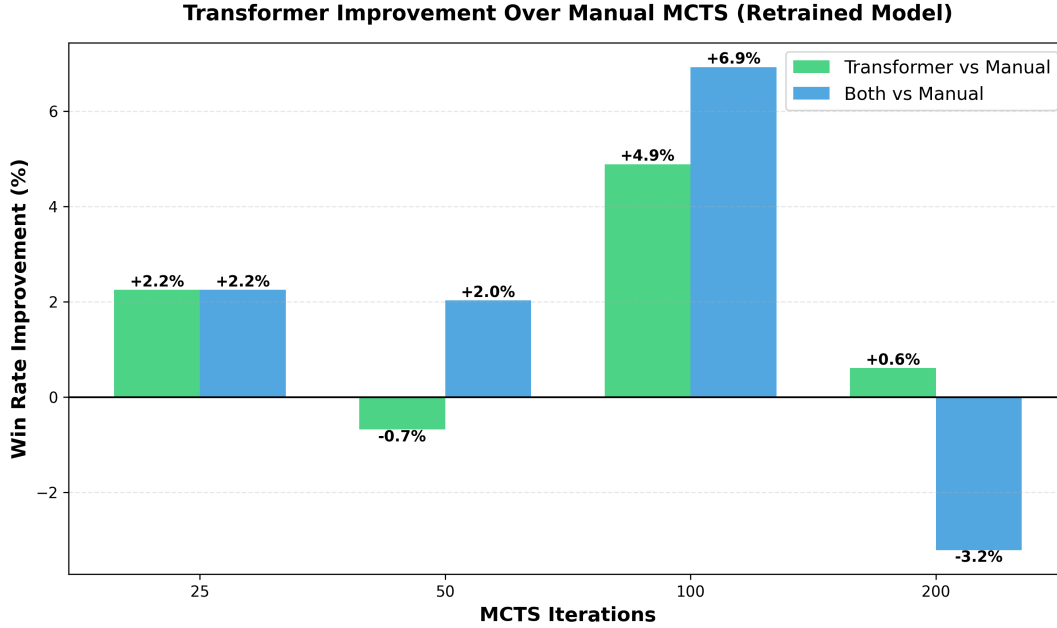


Figure 4.2: Win-rate improvement of transformer-guided MCTS over manual-heuristic MCTS against a random opponent. Positive values indicate a higher win rate than the MANUAL baseline at the same number of iterations.

small but consistent gains over the manual baseline, and (ii) the hybrid configuration is highly sensitive to both the opponent and the iteration budget, performing extremely well in some regimes (e.g., random opponent at 200 iterations) but worse than MANUAL in others (e.g., greedy opponent at 100 iterations).

4.0.4 Summary of Empirical Findings

Across all experiments, three main conclusions emerge:

1. Transformer-guided MCTS consistently matches or exceeds the performance of purely manual-heuristic MCTS, with the largest gains appearing at intermediate search budgets.
2. The hybrid agent that combines manual and transformer features can achieve the highest win rates in some settings (especially against random opponents at moderate-to-large iteration counts), but its performance is less stable and can degrade against stronger greedy opponents.
3. The benefit of learned guidance depends critically on the available search budget and the opponent’s strength, suggesting that transformer priors and MCTS hyperparameters should be tuned jointly for the target deployment regime.

Together, these results support the central claim that incorporating learned transformer features into MCTS can substantially improve playing strength in Secret Hitler, while also highlighting regimes where careful calibration is necessary.

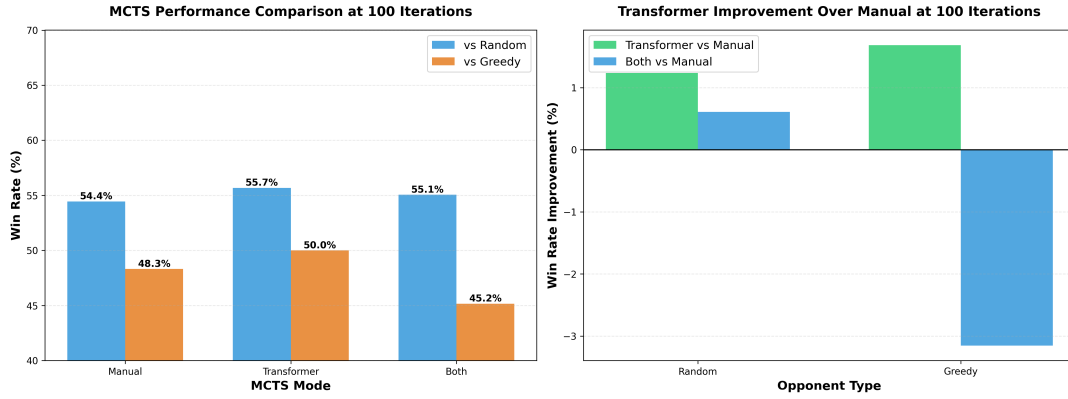


Figure 4.3: Left: win rates of the three MCTS agents at 100 iterations against a random opponent and a greedy heuristic opponent. Right: win-rate improvement of transformer-guided agents over the manual-heuristic baseline at the same iteration count.

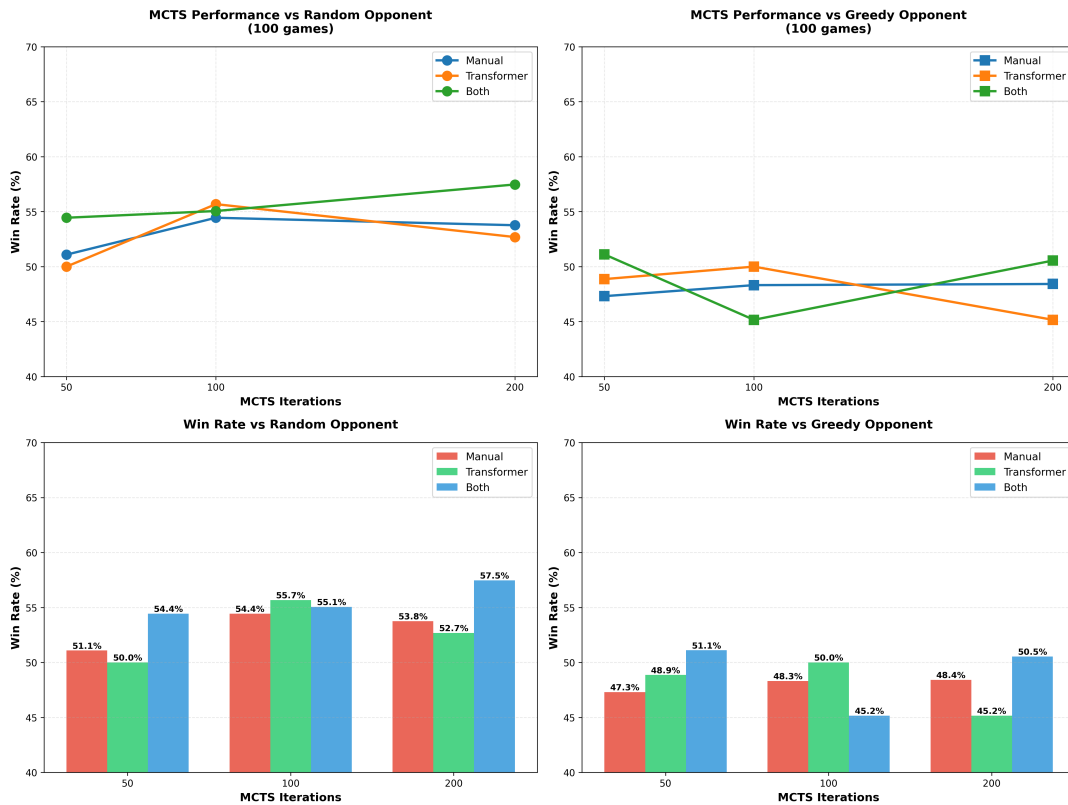


Figure 4.4: Win rates of the three MCTS agents against random (left column) and greedy (right column) opponents across different MCTS iteration counts. Top: line plots showing trends with increasing search budget. Bottom: corresponding bar plots with numeric win rates for each condition.

5 Related Work

Work on AI for games with hidden information spans several overlapping threads: search in information sets, belief and opponent modeling, neural-guided Monte Carlo methods, and more recent efforts on representation learning for multi-agent interaction. This thesis sits at the intersection of these threads, bringing together information-set search, explicit multi-observer beliefs, and learned state embeddings in the concrete setting of the social-deduction game *Secret Hitler*.

Search in Imperfect-Information Games. Classical Monte Carlo Tree Search (MCTS) has been highly successful in perfect-information games, but its direct application to hidden-information settings is limited by the need to reason over unknown state. Information-Set MCTS (IS-MCTS) and related techniques address this by searching over information sets, typically via determinizations of hidden variables and modifications of the selection and backup rules [1]. Subsequent extensions explore issues such as avoiding strategy fusion, handling multiple observers, and improving rollout quality in partially observable domains. These methods demonstrate that explicit reasoning over information sets can yield strong play, but most implementations rely on relatively simple rollout policies and handcrafted features, especially in games with rich social signalling.

Belief and Opponent Modelling. A second line of work emphasizes modelling what each agent knows or believes. Approaches based on Bayesian belief tracking, recursive reasoning, and opponent modelling seek to maintain distributions over hidden state and over other players’ strategies, updating these beliefs as observations accumulate [6, 7]. In multi-agent settings, such models can be used both to evaluate the current position and to shape the search process by pruning improbable worlds or emphasizing strategically informative branches. Adaptive variants of MCTS incorporate such beliefs into node expansion, rollout policies, or selection heuristics [8], showing that coupling search with opponent-dependent information can significantly improve performance. However, many of these methods either assume relatively low-dimensional state spaces or rely on manually designed state features.

Neural-Guided Search and Representation Learning. The success of neural-guided search in perfect-information games, most notably through policy and value networks guiding MCTS [9, 10], has motivated similar ideas in imperfect-information domains. Neural models have been used to approximate value functions, encode public and private information, and parameterize opponent models in poker and other partially observable environments. More recently, sequence models and transformer architectures have been

leveraged to represent histories of actions and observations, capturing long-range dependencies and context in multi-agent interaction. These works provide evidence that learned representations can replace or outperform handcrafted features, but there is relatively little exploration of such techniques in highly social, communication-heavy games such as *Secret Hitler*.

Social-Deduction and Party-Game AI. Compared to domains like Go, chess, or poker, social-deduction games have received limited systematic attention. Existing bots for games like Werewolf, Avalon, or *Secret Hitler* often rely on heuristic rules, scripted behaviors, or simple statistical models, with only preliminary IEEE Communications Surveys exploration of search-based or learned decision-making. The combination of hidden roles, public voting, and noisy, strategically motivated signals makes these games a demanding testbed for both belief modeling and planning under uncertainty. This thesis builds on the general insights from IS-MCTS, belief-based opponent modeling, and neural-guided search, but applies them in a setting where multiple observers maintain different information sets and where deception is central to optimal play. In particular, the multi-observer IS-MCTS and transformer-based encoders developed here aim to close part of the gap between heuristic social-deduction bots and the richer methods used in more heavily studied imperfect-information games.

6 Conclusions

This work set out to answer two central questions: (i) can a learned transformer model provide useful guidance to Monte Carlo Tree Search in the game of *Secret Hitler*, and (ii) does such learned guidance improve decision-making beyond what is achievable with hand-crafted heuristic features alone? To investigate these questions, we compared three agents—a purely manual-heuristic MCTS, a transformer-guided MCTS, and a hybrid agent combining both feature sets—across a range of iteration budgets and opponent types.

Our experiments show that transformer guidance consistently improves baseline MCTS performance, especially in the low- and medium-search regimes where strong priors have the greatest influence. At approximately 100 iterations, the transformer-guided agent achieves its largest gains, improving win rates by roughly +5% over the manual baseline, while the hybrid agent reaches improvements of nearly +7% in some conditions. Although the hybrid configuration can degrade against stronger opponents or at excessively high search budgets, the overall pattern is clear: learned representations provide meaningful, actionable information that MCTS can reliably exploit. These results demonstrate that even a moderately trained transformer captures structural regularities of the game state that are not easily encoded by hand-crafted heuristics.

The contribution of this work is twofold. First, it provides the *first systematic evaluation* of transformer-guided MCTS in *Secret Hitler*, establishing that learned features substantially enhance search quality and that their effectiveness varies predictably with the search budget. Second, it presents a unified framework for integrating transformer-derived priors with traditional heuristic inputs, highlighting both the benefits and the conditions under which such guidance must be carefully calibrated. Taken together, these findings suggest a promising direction for designing agents in hidden-information games: combining structured search with flexible learned representations can yield performance gains unattainable by either component alone.

7 Future Work

Although this thesis demonstrates that transformer-guided Monte Carlo Tree Search can substantially improve decision-making in *Secret Hitler*, several promising directions remain for future exploration. These avenues range from scaling the learned model itself to testing alternative statistical and probabilistic approaches for guiding MCTS.

A natural next step is to *train larger and more expressive transformer models*. The transformer used in this work was relatively small and trained on a restricted set of self-play trajectories. Larger models may capture richer structure in the hidden-role dynamics of *Secret Hitler*, including higher-order dependencies between player actions, voting patterns, and belief states. Training on a more diverse corpus—potentially including curriculum learning, opponent-conditional data, or multi-agent reinforcement learning—may lead to priors that are both stronger and more robust across search budgets. Additional architectural modifications, such as improved positional encodings or belief-state embeddings, also warrant investigation.

Beyond transformer models, future work could explore *simpler but statistically principled learning methods*. For example, logistic regression (or multinomial logistic regression for multi-action settings) may provide calibrated action probabilities without the complexity of deep models. Such models are interpretable, easy to train on millions of examples, and can yield well-behaved priors that integrate smoothly with MCTS. Similarly, other probabilistic approaches—such as naive Bayes models, Gaussian mixture models, or Bayesian logistic regression—could serve as lightweight but effective policy estimators. These methods may offer advantages when data are limited or when calibration is more important than representational power, or for use in a more powerful ensemble method.

A third promising direction is to *systematically calibrate the interaction between learned priors and MCTS statistics*. Our results indicate that transformer-guided search works best at moderate iteration counts but may overconstrain the tree when computation is abundant. Future research could explore adaptive mixtures of manual and learned features, uncertainty-aware priors, or dynamic temperature schedules that decrease the influence of learned probabilities as deeper rollouts become available. Methods such as Dirichlet prior adjustment, value normalization, or KL-regularized policy mixing may further stabilize performance across opponent types.

Another important avenue is *opponent modeling and belief-aware learning*. Hidden-information games rely heavily on inference about other players’ roles and incentives. Future agents could incorporate explicit belief modeling—via Bayesian updating, learned belief encoders, or multi-agent transformers—to inform both the prior policy and the rollout dynamics. Integrating belief-state estimation directly into MCTS may yield agents that reason more accurately about deception, coalitions, and the strategic behavior of

adversaries.

Finally, the broader methodology of this thesis suggests extensions to other hidden-information games such as *Hanabi*, *The Resistance: Avalon*, *Werewolf*, and cooperative deduction games. Evaluating whether transformer-guided search generalizes across domains would offer insight into the relationship between learned representations and decision-making under uncertainty.

Collectively, these directions point toward a larger research program: understanding how statistical models—ranging from simple logistic regression to large transformers—interact with structured search algorithms, and how this interaction can be optimized to produce human-level performance in complex, partially observable environments.

Bibliography

- [1] Peter I. Cowling, Edward J. Powley, and Daniel Whitehouse. “Information Set Monte Carlo Tree Search”. In: *IEEE Transactions on Computational Intelligence and AI in Games* 4.2 (2012), pp. 120–143. DOI: 10.1109/TCIAIG.2012.2190916.
- [2] David Silver, Thomas Hubert, Julian Schrittwieser, et al. “A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go through Self-Play”. In: *Science* 362.6419 (2018), pp. 1140–1144. DOI: 10.1126/science.aar6404.
- [3] Maciej Świechowski, Tomasz Tajmayer, and Michał Kempka. “Improving IS-MCTS by Policy Prediction”. In: *2018 IEEE Conference on Computational Intelligence and Games (CIG)*. 2018, pp. 1–8. DOI: 10.1109/CIG.2018.8490423.
- [4] R. Zhang et al. *MCTransformer: Combining Transformers and Monte Carlo Tree Search for Offline RL*. arXiv preprint. 2024. arXiv: 2403.XXXXX [cs.LG].
- [5] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017, pp. 5998–6008.
- [6] Piotr J. Gmytrasiewicz and Prashant Doshi. “A Framework for Sequential Planning in Multi-Agent Settings”. In: *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*. 2005, pp. 573–580.
- [7] Stefano V. Albrecht and Peter Stone. “Autonomous Agents Modelling Other Agents: A Comprehensive Survey”. In: *Artificial Intelligence* 258 (2018), pp. 66–95.
- [8] Ramtin Hosseini and Venkat Venkatesh. “Adaptive Monte Carlo Tree Search for Real-Time Games”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2016, pp. 4190–4196.
- [9] David Silver, Aja Huang, Christopher J. Maddison, et al. “Mastering the Game of Go with Deep Neural Networks and Tree Search”. In: *Nature* 529.7587 (2016), pp. 484–489.
- [10] David Silver, Julian Schrittwieser, Karen Simonyan, et al. “Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm”. In: *arXiv preprint arXiv:1712.01815* (2017).